

Field Agent App — Task Sheet

Companion to `PLAN_FIELD_AGENT_APP.md`. That document is the *why*: the wireframe assessment and decisions D0–D10. **This one is the *what to do*.**

Read the plan once, at the start. Then work from this sheet.

Every batch below is: what it depends on, the exact files, numbered steps, a **Done when** checklist you can verify yourself, and the repo-specific traps that will otherwise cost you an hour. If a step here contradicts the plan, the plan wins — tell whoever owns that batch.

Day-1 kickoff — 30 minutes, all three

Do this before anyone writes code. It is the only meeting in the week.

- Confirm lanes** (§4 of the plan): $A = \text{Amir} \cdot B = \text{Khalid} \cdot C = \text{Ali}$.
- Confirm the four seams** (§4 seam table) — especially that C builds the quote screens against a mock `QuoteOutput`, not against B's API.
- B starts the migration immediately**. A and C are both blocked on it, and only on it. It's fully specified in §3 of the plan; there is nothing to design.
- Agree the branch names** so nobody collides: `feat/agent-b0a-schema`, `feat/agent-b0b-scaffold`, `feat/agent-b1-dayview`, etc.

Then split up. The seam table exists so you don't need to talk again until Batch 9.

Conventions — everyone, every batch

Non-negotiable, and every one of them has bitten this repo before.

Imports

- Inside `apps/agent`, `@/` means `apps/agent/src/`. Nothing shared is ever reached with `@/` — it comes from `@clbipp/{ui,auth,core,database,pdf}`.
- Never import from `@prisma/client`**. Use `@clbipp/database`, which re-exports the client *and* every model type and enum.

What you'll actually import

NEED	IMPORT FROM
Prisma client + types + enums	<code>@clbipp/database</code>
Server Supabase client (session)	<code>@clbipp/auth/server</code> → <code>createClient()</code>
Service-role client (bypasses RLS)	<code>@clbipp/auth/admin</code> → <code>createAdminClient()</code>
Photo upload	<code>@clbipp/auth/storage</code> → <code>uploadFile</code> , <code>uploadFiles</code> , <code>buildObjectPath</code> , <code>MAX_FILE_BYTES</code>
Signed URLs for <code></code>	<code>@clbipp/auth/storage-server</code> → <code>createSignedUrl</code> , <code>createSignedUrls</code>
Realtime	<code>@clbipp/auth/realtime</code> → <code>subscribeToPickupEvents</code>
UI + tokens + lifecycle	<code>@clbipp/ui</code> → <code>AppShell</code> , <code>BottomAppBar</code> , <code>Card</code> , <code>Banner</code> , <code>Badge</code> , <code>ListRow</code> , <code>Timeline</code> , <code>buildStages</code> , <code>LifecycleHeader</code> , <code>RecoverySummary</code> , <code>LIFECYCLE_STAGES</code> , <code>STAGE_LABELS</code> , <code>isLifecycleStage</code> , <code>isStageBefore</code>
₹ formatting in a client component	<code>@clbipp/core/format</code> → <code>formatPaise</code>

△ **`formatPaise` in a client component must come from `@clbipp/core/format`, not `@clbipp/core`**. The package barrel re-exports `booking-actions` / `payment-actions`, so a value import from the barrel drags Prisma into the browser bundle. The subpath resolves to `documents.ts`, which imports nothing.

Money

- Integer paise everywhere. Never a float, never rupees, never a local `/100`.
- The engine returns **rupee floats** — convert at the boundary with `rupeesToPaise` (B builds it in Batch 4). Nowhere else.

Lifecycle

- Never re-declare the stage array in a screen. Use `isLifecycleStage` / `isStageBefore` / `STAGE_LABELS` from `@clbipp/ui`.
- The nine stages are locked. No migration adds one.

Writes

- Every agent lifecycle write goes through a "use server" action using `createAdminClient()`, which **re-verifies** `pickup.agentId === user.id` in code because the service role bypasses RLS.
- Read the session identity from `createClient().auth.getUser()`. **Never trust an id that came from the form.**
- Copy the shape from `apps/customer/src/app/(app)/handover/actions.ts` — it is the reference implementation.

Before you open a PR

```
npm run build      # type-checks every app + package
npm run test       # Vitest, all packages
npm run smoke      # logs in as a real seeded user and fetches every route
```

`npm run build` never renders a page with a session, so `npm run smoke` is the check that catches a server component throwing at request time. Add every new route to `ROUTES` in `scripts/smoke.mjs` as it lands.

Tests live in the package that owns the code, as `*.test.ts` next to the source. Apps hold no tests. If logic is worth testing, it belongs in `packages/core`, not in a screen.

Git: one feature = one small branch = one PR = one review. No stacked PRs, no direct pushes to `main`.

Batch 0a — Schema + seed · Khalid · ~0.4d

Depends on: nothing. **Blocks:** everything. Do it first, ship it same-day.

Edit

- `packages/database/prisma/schema.prisma`
- `packages/database/prisma/reset-demo.ts`

Steps

- Apply the §3 schema delta from the plan, exactly as written: `Pickup.agentFeePaise` · `Offer.acceptedAt` · `BatteryItem.damageVisual` / `damageLeakage` / `damageThermal` / `damageScore` / `pathway` / `traceId` · `WalletTxnKind + agent_fee` · new `CustodyBatch` · `Pickup.custodyBatchId`.
- `npm run db:migrate --workspace=@clbipp/database` — one migration, named `agent_app_v1`.
- Extend `reset-demo.ts`:
- The existing `agent@test` profile already has zone/vehicle/rating — leave it.
- Assign `agentId` on pickups so the agent has **at least one at each of** `scheduled`, `arrived`, `offered`, `collected`, and two further along for the watch-only timeline.
- Give the `scheduled` and `arrived` ones 2–3 `BatteryItem` rows each, **mixed categories** — at least one li-ion and one lead-acid. C cannot build the multi-item flow without this.
- Seed one `MarketPrices` row and one `Facility` row.
- `npm run reset-demo`, then open `npm run prisma studio` and eyeball it.

Done when

- ☐ `npm run build` green
- ☐ `npm run reset-demo` runs clean from a wiped DB
- ☐ `agent@test` has ≥ 1 pickup at each of the five stages above
- ☐ At least one pickup has a mixed-category `BatteryItem` set

Watch out

- `LIFECYCLE` in `reset-demo.ts` must stay in sync with `enum PickupStatus`, `LIFECYCLE_STAGES` in `packages/ui/src/tokens.ts`, and `pickupstatusSchema` in `packages/core`. You are not changing the list — just don't let it drift.
- `packages/database` **must not import** `packages/core` — the cycle breaks the generated client. If you need the CO2 table, restate it (there's already a test asserting the two agree).

Batch 0b — App scaffold + auth gate · Aamir · ~0.5d

Depends on: nothing (start in parallel with 0a). **Blocks:** C's Batch 3.

Create

- `apps/agent/src/proxy.ts`
- `apps/agent/src/app/(agent)/layout.tsx`
- `apps/agent/src/app/(auth)/login/page.tsx`
- `apps/agent/src/app/(agent)/page.tsx` — placeholder home
- Stub `page.tsx` for every route in §2 of the plan (a heading and nothing else)

Edit

- `apps/agent/package.json`, `next.config.ts` (mirror `apps/customer`'s `transpilePackages`)
- `scripts/smoke.mjs`

Steps

- Copy `apps/customer`'s `next.config.ts` and `tsconfig.json` shape into `apps/agent`. Packages ship raw TS and are compiled per-app via `transpilePackages` — there is no per-package build step.
- Write `apps/agent/src/proxy.ts`:

```
``ts export const proxy = createAuthMiddleware({ publicPaths: ['/login', '/auth'], homePath: '/', allowRoles: ['agent'], })`
```

 — `No` `onboardingPath` — agents don't self-sign-up (D6).

3. `(agent)/layout.tsx` : own the bottom-nav clearance the way `apps/customer/src/app/(app)/layout.tsx` does — the **layout** pays for the fixed bar, never the page. Screens pass `hideNav` to `AppShell` and nothing else.
4. Login page: email + password only. No Agent ID, no OTP (D6/W7).
5. Stub every `$2` route so links resolve before the screens exist.
6. Extend `scripts/smoke.mjs` to run against the agent app as `agent@test` .

Done when

- ☐ `npm run build` shows `f Proxy (Middleware)` for `apps/agent`
- ☐ `agent@test` can log in and land on `/`
- ☐ `business@test` (a customer) is **signed out** when hitting the agent app
- ☐ `npm run smoke` passes against the agent app
- ☐ Every `$2` route returns 200, even if the page is empty

Watch out

-  **proxy.ts** must live at `apps/agent/src/proxy.ts` , **not the project root**. Next's dev bundler silently never registers a root-level proxy file when `src/app` is in use, and an **unregistered auth guard fails OPEN**. This is the single most dangerous mistake available in this repo.
-  `packages/auth/src/middleware.ts` is **not** renamed and must not be — it's the `createAuthMiddleware` factory, an ordinary module, not a convention file.
- Don't give a page its own bottom padding. The layout owns clearance; per-page padding double-pads (the bug Batch 6.5 fixed on the customer side).

Batch 1 — Day view + job detail · Aamir · ~0.75d

Depends on: 0a + 0b.

Create

- `apps/agent/src/app/(agent)/page.tsx` — day view
- `apps/agent/src/app/(agent)/job/[id]/page.tsx`
- `apps/agent/src/app/(agent)/job/[id]/actions.ts`
- `apps/agent/src/lib/job-nav.ts`

Steps

1. Day view: Prisma read of pickups where `agentId === user.id` , scoped **in code**. Stats = assigned today / collected today / earned today. No "Avg margin" (it's a business figure, wrong on a contractor's home screen).
2. **No "New requests nearby" section** — jobs are pushed (D2).
3. `job-nav.ts` owns row → destination routing, mirroring `apps/customer/src/lib/pickup-nav.ts` . **Don't re-derive a row's destination inside a screen**.
4. Job detail: vendor, address, category, the customer's declared items and photos, agent fee. Actions:
5. **Open in Google Maps** — `https://www.google.com/maps/dir/?api=1&destination=<lat>,<lng>` from `Address.lat/lng`
6. **Call** — a plain `tel:` link
7. **Arrived** — server action, writes `arrived`
8. `actions.ts` — the **reference service-role action for this app**. C copies it, so make it clean and comment the ownership re-check.

Done when

- ☐ Day view lists only `agent@test` 's own pickups
- ☐ Tapping "Arrived" moves a `scheduled` pickup to `arrived` and writes a `StatusEvent` with `actorRole: 'agent'`
- ☐ Re-tapping "Arrived" is a **no-op, not an error** (idempotent)
- ☐ The action rejects a pickup belonging to a different agent
- ☐ Maps link opens; `tel:` link dials
- ☐ Both routes in `smoke.mjs` , `npm run smoke` green

Watch out

- `Address.lat/lng` are **nullable** — fall back to a text address search.
- Write `pickups.status` and the `status_events` row in one action. The status column is a denormalised cache of the event log; drift here is invisible until a timeline looks wrong.

Batch 2 — Safety checklist · Aamir · ~0.5d

Depends on: 1. This is the feature HR looks for first (W1).

Create

- `apps/agent/src/app/(agent)/job/[id]/safety/page.tsx`
- `apps/agent/src/app/(agent)/job/[id]/safety/actions.ts`

Steps

1. Checklist items, chemistry-aware, straight from the HR docs: terminals insulated · no puncturing · fire-safe crate · no mixed chemistry · PPE worn. Show lithium-specific items only when the pickup has a li-ion item.
2. Write a `SafetyChecklist` row (`items` Json, `passed` bool). The model already exists — no migration.
3. **Gate intake on it.** `/job/[id]/items` redirects back to `/safety` unless a `passed` checklist exists. The gate lives in the items page, server-side.
4. Show it as a completed step on the job detail screen once passed.

Done when

- ☐ Cannot reach `/job/[id]/items` without a passing checklist — verified by URL, not just by hiding the button
- ☐ All three HR-named items are present
- ☐ Re-submitting updates rather than duplicating (`pickupId` is `@unique`)
- ☐ Route in `smoke.mjs`, `npm run smoke` green

Watch out

- The UI is not the security boundary. Hiding the button is not the gate — the server-side redirect is.

Batch 3 — Multi-item intake · Ali · ~1.0d

Depends on: 0a + 0b. **The biggest UI batch — start it day 2.**

Create

- `apps/agent/src/app/(agent)/job/[id]/items/page.tsx` — the list
- `apps/agent/src/app/(agent)/job/[id]/items/[itemId]/page.tsx` — confirm one
- `apps/agent/src/app/(agent)/job/[id]/items/actions.ts`
- `apps/agent/src/app/(agent)/job/[id]/scan/page.tsx` — optional, do last

Steps

1. **Item list** — the spine of the flow. Every `BatteryItem` on the pickup, each showing declared vs confirmed state, plus a running total (count + kg). "Continue to quote" is disabled until every item is confirmed.
2. **Item confirm** — per item, the agent sets: category, chemistry (`BatteryType`), weighed kg, condition, photos. These write the *agent-confirmed half* of `BatteryItem`: `chemistry`, `confirmedWeightKg`, `confirmedCondition`, `agentPhotoUrls`, `recordedBy`, `recordedAt`. **Never overwrite the customer-declared fields** — both halves are evidence.
3. **Branch on chemistry:** li-ion (`li_ion_nmc` / `li_ion_lfp` / `li_ion_nca`) → "Continue to damage rubric". Everything else → straight to price. This is D1 and it is the whole reason this batch is big.
4. **Photos** upload through a "use server" action using the service role — agent photos are written by the service role per `storage-policies.sql`. Bucket `pickup-photos`; build the path with `buildObjectPath`. Check `MAX_FILE_BYTES` client-side before sending — the bucket limit is a backstop that fails *after* a full upload, which is a bad thing to learn on mobile data.
5. QR scan last, and only if there's time. Manual entry is the primary path.

Done when

- ☐ A pickup with 3 mixed-category items can be fully confirmed
- ☐ Li-ion items route to the rubric; lead-acid items skip it
- ☐ Photos land in `pickup-photos` and render back via `createSignedUrl`
- ☐ Leaving mid-way and returning keeps confirmed items confirmed
- ☐ Customer-declared fields are unchanged after confirming
- ☐ Routes in `smoke.mjs`, `npm run smoke` green

Watch out

- ⚠ **If this overruns by more than half a day, stop and fall back** to one li-ion item per pickup, keeping the loop only for the simple path. Contained retreat, decided on the spot — say so in the group chat, don't absorb it silently. (§5 risk 2.)
- Photos are evidence of one consignment — never copy them between items.

Batch 4 — Engine + pricing · Khalid · ~1.2d

Depends on: 0a. Pure logic — no UI, no app files.

Create

- `packages/core/src/agent-fee.ts` + `agent-fee.test.ts`
- `packages/core/src/market.ts` — `MarketPrices` row → engine `MarketData`
- `packages/core/src/mock-data.ts` — a mock `QuoteOutput`. **Ship this on day 2; C is building against it.**
- `apps/agent/src/app/api/quote/route.ts`

Edit

- `packages/decision-engine/src/decisionEngine/layers/intake.ts`
- `packages/decision-engine/src/decisionEngine/{types,defaults}.ts`

- `packages/core/src/documents.ts` — add `rupeesToPaise`

Steps

1. **Fix defect 1 (market freshness).** `MARKET_FRESHNESS_MAX_HOURS = 24` is a module constant in `intake.ts`. Move it to a `Config` field. Then have `market.ts` stamp `snapshot_timestamp` at read time in simulated mode — same posture as `PAYMENTS_MODE`. **A row seeded Monday otherwise breaks the demo Tuesday.**
2. **Fix defect 2 (trace_id).** `traceCounter` is a module-level in-memory counter; on Vercel every cold start resets it, so IDs collide and the audit trail the "Why" screen is built on quietly corrupts. Accept a caller-supplied `trace_id`, derive it from the pickup/item id, keep the counter as a test-only fallback.
3. `rupeesToPaise` — round **half-up at the paise level**. Never round-trip back to a float.
4. `agent-fee.ts` — `base_fee + per_km_rate × distance_km`, rounded to the rupee, constants in that one file (D3).
5. `POST /api/quote` — validate input, call `computeQuote`, persist a `PathwayDecision`, return `QuoteOutput`. Map `EngineValidationError` → 422 and `StaleMarketDataError` → 503, as the engine's own header instructs.
6. Non-li-ion items price through the **existing** `estimateQuote` in `packages/core/src/booking.ts`. Don't write a second pricing path.
7. Tests for all of it, in `packages/core` / `packages/decision-engine`.

Done when

- ☐ `npm run test` green — the engine's existing **20 tests still pass**
- ☐ New tests cover both defect fixes
- ☐ `POST /api/quote` returns a valid `QuoteOutput` for a seeded li-ion item
- ☐ A >24h-old market row no longer throws in simulated mode
- ☐ Two concurrent quotes get **different** trace IDs
- ☐ `mock-data.ts` exports a realistic `QuoteOutput` (shipped day 2)

Watch out

- 🚫 **A change that moves a price must say so explicitly in the PR.** Silent economics drift is the one failure here nobody notices until a demo.
- Fix defects and anything the HR documents contradict. **Don't refactor the engine because it could be nicer** — 1,476 lines, 20 tests, live pricing surface, seven-day week.

Batch 5a — Quote screens + offer · Ali · ~0.75d

Depends on: 3. **Not on Batch 4** — build against `mock-data.ts`.

Create

- `apps/agent/src/app/(agent)/job/[id]/items/[itemId]/damage/page.tsx`
- `.../[itemId]/computing/page.tsx`
- `.../[itemId]/result/page.tsx` · `result/breakdown/page.tsx` · `result/why/page.tsx`
- `apps/agent/src/app/(agent)/job/[id]/offer/page.tsx` + `actions.ts`

Steps

1. **Damage rubric** — Visual 0.40 / Leakage 0.35 / Thermal 0.25, each 0–3 with a photo slot. Matches `DamageScores` in the engine exactly. Show the live weighted score and the routing rule (≤ 1.5 all pathways · 1.6–2.5 Refurbish/Recycle · > 2.5 forces Recycle). Build the forced-Recycle state too.
2. **Computing** — the six-layer stepper. Honest: it reflects the real layers.
3. **Result: Verdict / Breakdown / Why.** Render `QuoteOutput` 1:1. This is the deliberate **inverse** of the vendor rule — the agent sees full revenue, every cost line, net value, margin, and the $P_{\min}/P_{\text{rec}}/P_{\max}$ band. **Cut the "AI explanation" button** — there is no `/api/explain`.
4. **HOLD and REVIEW branches.** HOLD blocks presenting an offer. "Escalate to admin" must actually **write** something — a flag and a note on the pickup — not navigate home.
5. **Offer roll-up screen** — the multi-item consequence: per-item prices sum into one `Offer` for the pickup. Creates the `Offer` row and writes `offered`. The wireframe had no such screen because it assumed one battery.
6. When B's `/api/quote` lands, swap the mock import. One line.

Done when

- ☐ All three result tabs render from a `QuoteOutput`
- ☐ HOLD cannot reach the offer screen; REVIEW can, with a warning
- ☐ "Escalate to admin" persists something visible in the DB
- ☐ Presenting creates exactly one `Offer` and writes `offered`
- ☐ `Offer.estimatedPrice` is **paise**, and every ₹ uses `formatPaise`
- ☐ Routes in `smoke.mjs`, `npm run smoke` green

Watch out

- ⚠ Everything on these screens is **agent-only**. None of it may leak to a vendor surface — `/offer` and `/offer-breakdown` in the customer app stay price + qualitative rationale, weight-only, no margins, no recovery rate.
- `estimatedPrice` is `Int` paise; the engine speaks rupee floats. Convert once, with `rupeesToPaise`.

Batch 5b — Cross-app seam · Aamir · ~0.4d

Depends on: 0a. Highest-risk correctness item in the plan (D7/W9).

Edit

- `apps/customer/src/app/(app)/handover/actions.ts`
- `apps/customer/src/app/(app)/handover/page.tsx`
- `apps/customer/src/app/(app)/offer/page.tsx`

Steps

1. `acceptOffer` currently jumps `offered → collected`. **Stop it writing `collected`**. It now sets `Offer.acceptedAt` and leaves status at `offered`. A vendor cannot mark their own battery collected — only the agent on site can.
2. Make `/handover` a **POST**, not a GET-time mutation. It currently mutates on page load, which is why it is excluded from `npm run smoke`. Fix that here and **add it back to `ROUTES`**.
3. Update the customer's copy: accepting now means "agent will collect", not "collected".

Done when

- ☐ Accepting an offer sets `acceptedAt` and leaves status `offered`
- ☐ Loading `/handover` mutates **nothing**
- ☐ `/handover` is back in `smoke.mjs` and passes
- ☐ The customer app's existing flows still work end to end
- ☐ `npm run build` + `npm run test` + `npm run smoke` green **for both apps**

Watch out

- 🚫 This touches a **merged, deployed** app. Small PR, own branch, nothing else bundled with it. Coordinate with Khalid if the customer app is mid-deploy.

Batch 6 — Collect · Ali · ~0.75d

Depends on: 5a + 5b.

Create

- `apps/agent/src/app/(agent)/job/[id]/collect/page.tsx` + `actions.ts`
- `apps/agent/src/app/(agent)/job/[id]/receipt/page.tsx`

Steps

1. Gate the screen on `Offer.acceptedAt` being set (5b writes it). If the vendor hasn't accepted, say so — don't show a collect button.
2. Capture: battery photo, sealed-packaging photo, drop-off slot, contact confirm, and a **signature** (canvas → PNG → `pickup-photos` via the service role).
3. On confirm, in **one action**: write `collected` + the `StatusEvent` (with GPS from `navigator.geolocation`) + a `PickupReceipt` row + a `WalletTxn` of kind `agent_fee` for the agent, and update the cached `profiles.wallet_balance_paise` **in the same transaction**.
4. Receipt screen: pickup id, wallet credit, share action, "Go to hub drop-off".
5. Vendor-declines branch: closes the request, collects nothing.

Done when

- ☐ Collect is unreachable until the vendor has accepted
- ☐ Confirming writes status + event + receipt + wallet txn **atomically**
- ☐ `wallet_balance_paise` equals `sum(wallet_txns.delta_paise)` afterwards
- ☐ Re-submitting doesn't double-credit the wallet
- ☐ Declining closes the request and collects nothing
- ☐ Routes in `smoke.mjs`, `npm run smoke` green

Watch out

- 🚫 **Wallet writes must be idempotent**. A double-tap on a flaky connection paying an agent twice is the worst bug available in this batch. Follow `settlePayment` in `packages/core/src/payment-actions.ts` — it's already idempotent, atomic and ownership-scoped.
- `WalletTxn` is the source of truth; the profile column is a cache. Always write both together.

Batch 7a — Hub drop-off UI · Ali · ~0.6d

Depends on: 6.

Create

- `apps/agent/src/app/(agent)/dropoff/page.tsx` — batch select
- `apps/agent/src/app/(agent)/dropoff/confirm/page.tsx` + `actions.ts`
- `apps/agent/src/app/(agent)/dropoff/[batchId]/page.tsx` — receipt

Steps

1. **Select:** every pickup at `collected` with no `custodyBatchId`, checkable, with a running item count and weight.
2. **Confirm:** pick a `Facility`, show the batch summary, capture GPS + timestamp, type the receiving staff name, capture a staff signature.
3. Write one `CustodyBatch` and set `custodyBatchId` on each pickup in it.
4. **Receipt** screen renders the batch and links to the PDF (Khalid's 7b).
5. On screen, state plainly that the hand-off is **agent-attested** — there is no hub-staff app, so the receiving name is typed, not authenticated. Being honest in a chain-of-custody document matters more than looking complete.

Done when

- ☐ Only `collected`, un-batched pickups are selectable
- ☐ Confirming creates one `CustodyBatch` and stamps every pickup in it
- ☐ Those pickups stop appearing in the next drop-off selection
- ☐ GPS + timestamp are persisted
- ☐ Routes in `smoke.mjs`, `npm run smoke` green

Batch 7b — Chain-of-custody PDF · Khalid · ~0.4d

Depends on: 0a (the `CustodyBatch` model). Pure render — no UI.

Create

- `packages/pdf/src/templates/custody.tsx`
- `apps/agent/src/app/api/documents/custody/[batchId]/route.ts`

Steps

1. Template alongside the three existing ones, same `theme.ts`, same shape. Contents: batch no, agent, facility, per-pickup lines, total weight + count, GPS, timestamp, receiving staff.
2. Route follows the **Batch 8 document pattern exactly**: ownership-scoped read, then **stream the bytes**. Do not mint a signed URL (that's for photos, which need a URL for `<img>`).
3. Generate lazily on first download, then cache. **pdfUrl** holds a storage PATH, not a URL — same as every other document in this repo.
4. Derive the batch number the way `certificateNumber` / `invoiceNumber` are derived in `packages/core/src/documents.ts` — no new column.

Done when

- ☐ The PDF renders for a seeded batch
- ☐ Another agent's batch is rejected
- ☐ Second download serves the cached copy
- ☐ Route in `smoke.mjs`'s `DOCUMENT_ROUTES`, `npm run smoke` green

Batch 8 — Track, history, profile · Aamir · ~0.75d

Depends on: 0a (seeded data — not on C's batches).

Create

- `apps/agent/src/app/(agent)/pickups/page.tsx` · `pickups/[id]/page.tsx` · `pickups/[id]/map/page.tsx`
- `apps/agent/src/app/(agent)/history/page.tsx` · `history/[id]/page.tsx`
- `apps/agent/src/app/(agent)/profile/page.tsx`

Edit

- `supabase/policies.sql`

Steps

1. `/pickups` — active jobs with a bulk drop-off shortcut.
2. `/pickups/[id]` — render **lifecycle-view.tsx** from `@clbipp/ui` (`buildStages`, `LifecycleHeader`). Do **not** write the wireframe's Collected → Transit → Warehouse → Refurb/QA → Done timeline; that vocabulary doesn't exist (D5/W4). Add the "your role ends at drop-off" lock banner.
3. **Realtime:** Δ RLS on `status_events` is vendor-scoped, so an agent's browser subscription **silently receives nothing**. Add an agent-scoped SELECT policy to `supabase/policies.sql` (~6 lines, mirroring the vendor one but joining on `pickups.agent_id`), then use `subscribeToPickupEvents`. Prototyping in the dashboard is fine — **the final version must land in the repo file**.
4. Map — Leaflet + OSM, static. No turn-by-turn (D4).
5. History — filterable, and **rows link to a real detail view** (the wireframe's link to themselves).
6. Profile — wallet balance, earnings from `WalletTxn`, stats, read-only safety-training status from `safetyTrainedAt`, sign out. **Delete "Cash out" and "Notifications" unless you have time to make them work** — dead buttons are worse than absent ones.

Done when

- ☐ Timeline renders through the shared component, not a local copy
- ☐ An agent receives a realtime ping on their own pickup's new event
- ☐ The new policy is in `supabase/policies.sql`, not only in the dashboard

- ☐ History rows open a real detail view
- ☐ Wallet balance matches the sum of the agent's `WalletTxn` rows
- ☐ Routes in `smoke.mjs`, `npm run smoke` green

Batch 9 — Deploy + verification · Khalid, then all three · ~0.5d

Depends on: everything.

Steps

1. New Vercel project for `apps/agent`, GitHub-synced. **Build command must go through turbo** — the generated Prisma client is gitignored.
2. Env vars + Supabase redirect URLs for the new domain. Follow `docs/DEPLOY.md` and `docs/HANDOVER_KHALID_2026-08-12.md` — same shape as the customer app.
3. `npm run build && npm run test && npm run smoke` for **both** apps.
4. **Then all three, together — the one manual pass**, on a real handset:
5. Customer books → agent sees it assigned → arrives → safety checklist → confirms items → quote → presents offer → **customer accepts on a second device** → agent collects → hub drop-off → both apps show the right state.
6. That cross-app hop is the part no automated check covers. Do it last, do it properly, and do it with time left to fix what it finds.

Done when

- ☐ Both apps deployed and reachable
- ☐ `agent@test` cannot enter the customer app, and vice versa
- ☐ The full two-device journey above completes
- ☐ Any gaps found are written down, owner named, before anyone goes home

If the week slips

Cut in this order, and **say so in the group chat** — a decided cut is fine, a half-finished batch is not:

1. Batch 8 extras — map, then history detail, then wallet cash-out
2. QR scan (Batch 3)
3. HOLD / REVIEW branches (Batch 5a)

Batches 0–7 minus those are the demo path and cannot be cut.